

REQUIREMENT 1: STATE-SPACE SEARCH FORMULATION

To apply search algorithms, the Sokoban problem is formulated as a State-Space Search Problem with 5 core components. The team strongly focused on Data Structure design to optimize memory allocation and execution time.

1. State Space (S)

S is the set of all possible states in the game environment. For optimization purposes, the team segregated the data into two categories:

Static Environment: The Walls (%) and Goals (D) remain unchanged throughout the game. They are stored using the Set() data structure. Thanks to the Hash Table mechanism, the time complexity for collision lookup is dramatically reduced to $O(1)$ instead of $O(n)$ as seen in standard Lists.

Dynamic State: The coordinates of the Agent (A) and the Boxes (B). Since the box coordinates must be stored in the visited set to track explored states, the team cast the list of boxes into an immutable Tuple combined with the sorted() function. This guarantees the uniqueness of the box configurations and strictly prevents unhashable memory errors.

2. Initial State (S_0)

The initial state is parsed directly from the input text file example_map.txt. The (x, y) coordinate system is established with the origin at the top-left corner, mapping each character to its corresponding entity (% , A, B, D, C).

3. Actions (A)

Given a valid state, the agent can execute 4 movement actions:

$A = \{ \text{North, South, East, West} \}$ (Corresponding to the vectors: (0, -1), (0, 1), (1, 0), (-1, 0)).

4. Transition Model ($T(s, a)$)

The transition function `get_successors(state)` takes the current state as input and returns all valid successor states. It strictly validates physical constraints:

Case 1: The next cell is an empty floor or a goal → The agent moves successfully.

Case 2: The next cell contains a Box → The system evaluates the cell immediately behind the box. If that cell is empty/goal, the agent moves forward, and the box is pushed in the same direction.

Case 3 (Invalid): The next cell is a Wall, OR it is a Box but the cell behind it is a Wall/another Box → The action is blocked, and the move is discarded.

5. Goal Test (G)

A state is considered a goal state when the coordinate set of all Boxes perfectly matches the Goals set. The team utilized the Tuple Unpacking technique (`_, boxes = state`) to discard the agent's coordinates during the test, focusing solely on comparing box positions to conserve processing resources.

REQUIREMENT 2: SEARCH ALGORITHMS AND HEURISTIC FUNCTION

The team implemented two pathfinding algorithms: Uniform Cost Search (UCS) and A-Star (A*). Both algorithms leverage the Priority Queue data structure via Python's heapq library.

1. Priority Queue Stability Enhancement

During the implementation of UCS and A*, an issue arises when two states share the exact same cost; the system attempts to compare the inner state tuples, leading to fatal crashes. To resolve this, the team injected a counter variable (tracking the sequence of generated nodes) into the tuple: (cost, counter, state, path). This variable acts as a strict "Tie-breaker", ensuring the algorithm's absolute stability.

2. Proposed Heuristic Function for A*

The project requirements strictly prohibit the use of Euclidean and Manhattan distances. Therefore, the team proposed a more robust Heuristic function based on 2 factors: Chebyshev Distance and Corner Deadlock Pruning.

Admissible Estimation (Chebyshev Distance):

The team utilized the Chebyshev Distance to estimate the minimum number of steps required to move a Box to a Goal. Formula:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Since diagonal movements are illegal in Sokoban, the estimated steps using Chebyshev will always be less than or equal to the actual required movement steps. Consequently, this Heuristic function flawlessly satisfies the Admissibility property.

Deadlock Pruning Technique:

The team developed a geometric function `is_deadlock()` to detect unresolvable states (dead ends). Specifically, if the system identifies a box (not yet on a goal) tightly trapped by two walls on both the horizontal and vertical axes (forming a 90-degree corner), the Heuristic function immediately returns an Infinity value ($h(n) = \infty$). Upon receiving an infinity value, the A* algorithm executes aggressive Pruning, permanently discarding this fatal branch from the priority queue. This enhancement drastically reduces the size of the frontier set, accelerating the map-solving speed exponentially.